

Exam 2

● Graded

Student

Cayden Gosseck

Total Points

79 / 100 pts

Question 1

Java Code

23.5 / 29 pts

1.1 Java Code

18.5 / 24 pts

Class Header and Data Members

✓ - 1.5 pts Data members missing visibility

Constructors

✓ - 2 pts Constructor(s) should use mutator methods

toString() method

✓ - 2 pts toString() should properly retrieve values for all properties

1.2 Java Code

5 / 5 pts

✓ - 0 pts Correct/No adjustments

Question 2

Packages

6 / 6 pts

2.1 Packages

2 / 2 pts

✓ - 0 pts Correct

2.2 Packages

2 / 2 pts

✓ - 0 pts Correct

2.3 Packages

2 / 2 pts

✓ - 0 pts Correct

Question 3

Java Code

7.5 / 9 pts

3.1 Java Code

3 / 3 pts

✓ - 0 pts Correct

3.2 Java Code

1.5 / 3 pts

✓ - 1.5 pts Correct Yes/No, incorrect justification

3.3 Java Code

3 / 3 pts

✓ - 0 pts Correct

Question 4

Vocabulary/Theory

3 / 3 pts

✓ + 3 pts Correct

Question 5

Vocabulary/Theory

3 / 3 pts

✓ - 0 pts Correct

Question 6

Reading UML/Java Code

9.5 / 11 pts

6.1 Reading UML/Java Code

3 / 3 pts

✓ - 0 pts Correct

6.2 Reading UML/Java Code

3 / 3 pts

✓ - 0 pts Correct

6.3 Reading UML/Java Code

3.5 / 5 pts

✓ - 1.5 pts Missing/incorrect composed Engine object

Question 7

Vocabulary/Theory

1.5 / 3 pts

✓ - 1.5 pts Missing one correct answer

Question 8

Vocabulary/Theory

3 / 3 pts

✓ + 3 pts Correct

Question 9

Vocabulary/Theory

0 / 3 pts

✓ + 0 pts Incorrect

Question 10

Vocabulary/Theory

3 / 3 pts

✓ + 3 pts Correct

Question 11

Vocabulary/Theory

3 / 3 pts

✓ + 3 pts Correct

Question 12

Vocabulary/Theory

0 / 3 pts

✓ + 0 pts Incorrect

Question 13

Reading UML/Java Code

3 / 3 pts

✓ - 0 pts Correct

Question 14

Vocabulary/Theory

0 / 3 pts

✓ - 3 pts Incorrect

Question 15

Vocabulary/Theory

3 / 3 pts

✓ + 3 pts Correct

Question 16

Vocabulary/Theory

3 / 3 pts

✓ + 3 pts Correct

Question 17

Vocabulary/Theory

3 / 3 pts

✓ + 3 pts Correct

Question 18

Vocabulary/Theory

1 / 3 pts

✓ - 2 pts Missing two correct options

Question 19

Vocabulary/Theory

3 / 3 pts

✓ + 3 pts Correct

Q1 Java Code

29 Points

Answer the following questions.

Q1.1 Java Code

24 Points

Create a class named `Motorcycle` with the following data members and methods.

`Motorcycle` is a subclass of another class named `TwoWheeledVehicle`. All `Motorcycle`s have an `Engine`, and `Engine`s can be purchased on their own and installed into an existing `TwoWheeledVehicle`.

`TwoWheeledVehicle` contains the following data members and their respective accessors/mutators: `frontWheelSize` as a `String`, `rearWheelSize` as a `String`, `powered` as a `boolean`, and `year` as an `int`.

`Motorcycle` expands `TwoWheeledVehicle` by adding the following data fields and their respective accessors/mutators: `make` as a `String`, `model` as a `String`, and `engine` as an `Engine`.

`Engine` contains the following data members and their respective accessors/mutators: `displacement` as a `int`, and `manufacturer` as a `String`.

In Java, implement **only** the following in the `Motorcycle` class (assume all data members and accessors/mutators already exist in all classes except any requested to be implemented below):

- Class header based on `Motorcycle`'s relationship to `TwoWheeledVehicle`.
- All data members with appropriate visibility.
- A default constructor that sets the `powered` data member to `true` and `year` to 2023 through the `TwoWheeledVehicle` class.
- A convenience constructor that receives and uses values for `make`, `model`, and `Engine` as parameters. The default constructor of `Motorcycle` should be called from this constructor to set the default values for `powered` and `year`. This constructor should also initialize the following default values: `frontWheelSize`: "120/70-17", `rearWheelSize`: "190/55-17" using superclass mutator methods.
- Accessor and mutator methods for the `make` data member. A `make` that is not 3 or more characters should throw an `InvalidMakeException` (assume the class exists).
- Implement a `toString()` method that returns all information about the `Motorcycle`. Assume that the `TwoWheeledVehicle` and `Engine` class's `toString()` methods provide formatted information about data members that belong to them. Provide descriptive text within this method.

For all items, use best practices according to Java/OOP convention.

```
public class Motorcycle extends TwoWheeledVehicle {  
  
    // data  
    String make, model;  
    Engine engine;  
}
```

```
// def. constructor
public Motorcycle() {
    setPowered(true);
    setYear(2023);

}

// conv. constructor
public Motorcycle(String make, String model, Engine engine) {

    this();

    try {
        setMake(make);
    } catch (invalidMakeException e) {
        System.out.println("Error! Invalid Make!")
    }

    this.model = model;
    this.engine = engine;

    setFrontWheelSize("120/70-17");
    setRearWheelSize("190/55-17");
}

// if make < three characters, throw InvalidMakeException
public void setMake(String make) throws InvalidMakeException {

    if(make.length() < 3) {
        throw new InvalidMakeException();
    } else {
        this.make = make;
    }

}

public String getMake() {
    return this.make;
}

// to string returns motorcycle, engine, and TwoWheeledVehicle info
@Override
```

```
public void toString() {  
    super.toString();  
    String motorcycleInfo = String.format("Make: %s Model: %s Engine Info: ",  
        getMake(), this.model);  
    return motorcycleInfo + engine.toString() + super.toString();  
}  
  
}
```

Q1.2 Java Code

5 Points

Write a main method that:

- Creates a collection containing:
 - two `Motorcycle`s: one made in 2020 with a 250cc engine displacement, another made in 2021 with a 375cc displacement
 - and one `TwoWheeledVehicle` made in 2022.
- Using iteration, output information about every item in the collection.
- Use exception handling and print an appropriate error message for any `Exception`s that occur (refer to instructions on creating the `Motorcycle` class).

Assume best practices according to Java/OOP convention are being followed for all classes.

```
public static void main(String[] args) {  
  
    // create and initialize list and engines, set displacement  
    ArrayList<TwoWheeledVehicle> vehicles = new ArrayList<>();  
    Engine e1 = new Engine();  
    e1.setDisplacement(250);  
    Engine e2 = new Engine();  
    e2.setDisplacement(375);  
  
    // try catch for InvalidMakeException  
    try {  
  
        // create and initialize objects  
        Motorcycle m1 = new Motorcycle("make", "model", e1);  
        Motorcycle m2 = new Motorcycle("make", "model", e2);  
        TwoWheeledVehicle tw1 = new TwoWheeledVehicle();  
  
        // set the years  
        tw1.setYear(2022);  
  
        m1.setYear(2020);  
        m2.setYear(2021);  
  
        // add to vehicle collection  
        vehicles.add(m1);  
        vehicles.add(m2);  
        vehicles.add(tw1);  
    }  
}
```



```
// print all the info from vehicle collection

for(int i = 0; i < vehicles.size(); i++) {
    System.out.println(vehicles.get(i).toString());
}

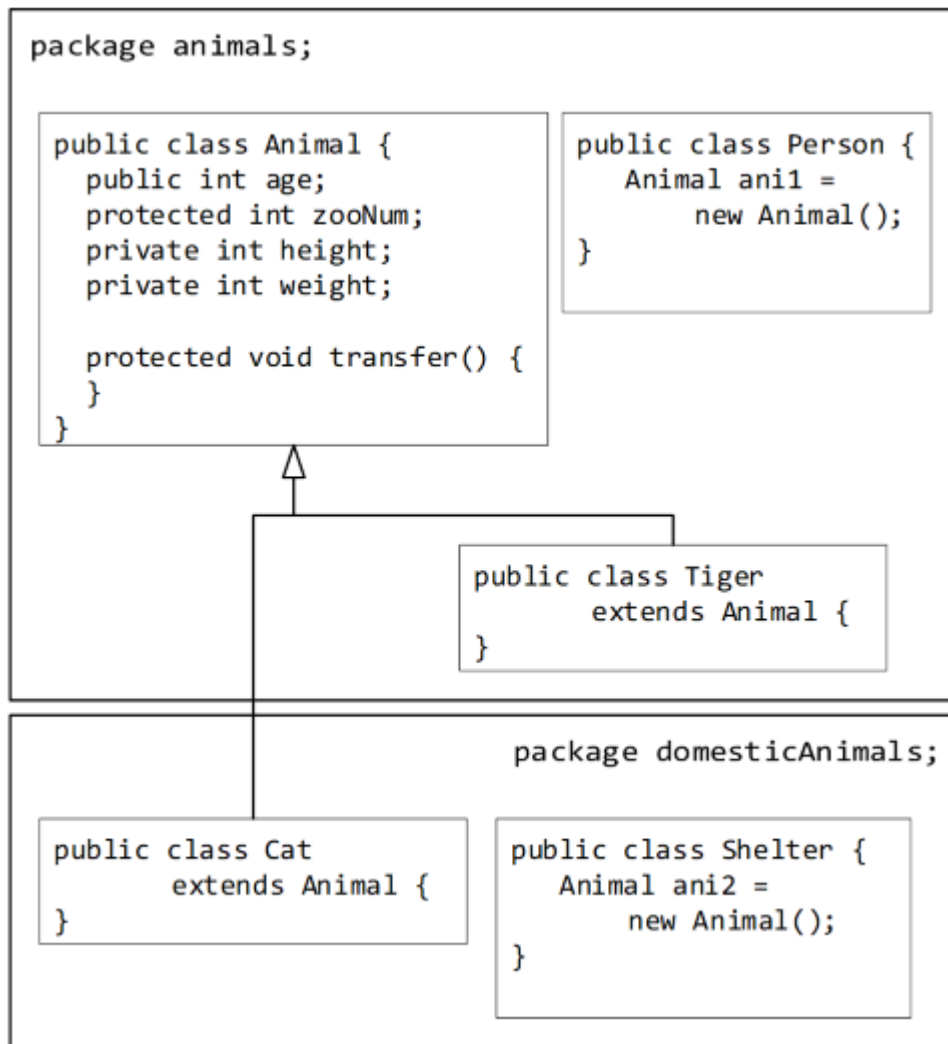
} catch(InvalidMakeException e) {

    System.out.println("Error! Invalid Make. Make must be 3 or more
    characters.");

    }
}
```

Q2 Packages

6 Points



Use the above image to answer the following questions.

Q2.1 Packages

2 Points

In class `Cat`, can `super.age` be accessed directly?

- ☐ No, private visibility
- ☒ Yes, public visibility
- ☐ No, different package
- ☐ No, protected visibility
- ☐ Yes, protected visibility

Q2.2 Packages

2 Points

In class `Tiger`, can `super.transfer()` be invoked?

- ☐ No, different package
- ☐ Yes, public visibility
- ☒ Yes, protected visibility
- ☐ No, private visibility
- ☐ No, protected visibility

Q2.3 Packages

2 Points

In class `Cat`, can `super.height` be accessed directly?

- ☐ Yes, public visibility
- ☐ No, protected visibility
- ☐ Yes, protected visibility
- ☐ No, different package
- ☒ No, private visibility

Q3 Java Code

9 Points

Answer the following questions.

Q3.1 Java Code

3 Points

Assume that `Sock` is the superclass of `WoolSock`. Will the following casting attempt(s) work and why? Consider all of the code presented.

```
Sock s = new WoolSock();  
WoolSock ws = s;
```

- ☐ No, the data types must match exactly for this to work
- ☒ No, need an explicit cast for this to succeed
- ☐ Yes, an implicit cast allows this
- ☐ Yes, an explicit cast allows this
- ☐ No, casting can only work from subclass to superclass
- ☐ Yes, a superclass variable can hold a subclass instance

Q3.2 Java Code

3 Points

Assume that `Sock` is the superclass of `WoolSock`. Will the following casting attempt(s) work and why? Consider all of the code presented.

```
Sock s2 = new WoolSock();
```

- ☐ No, need an explicit cast for this to succeed
- ☐ No, casting can only work from subclass to superclass
- ☒ Yes, a superclass variable can hold a subclass instance
- ☐ Yes, an explicit cast allows this
- ☐ Yes, an implicit cast allows this
- ☐ No, the data types must match exactly for this to work

Q3.3 Java Code

3 Points

Assume that `Sock` is the superclass of `WoolSock`. Will the following casting attempt(s) work and why? Consider all of the code presented.

```
Sock s3 = new WoolSock();  
WoolSock ws3 = (WoolSock)s3;
```

- ☐ Yes, an implicit cast allows this
- ☐ Yes, a superclass variable can hold a subclass instance
- ☐ No, need an explicit cast for this to succeed
- ☐ No, the data types must match exactly for this to work
- ☐ No, casting can only work from subclass to superclass
- ☒ Yes, an explicit cast allows this

Q4 Vocabulary/Theory

3 Points

Casting from a ____ to a ____ is considered a downcast.

- ☒ superclass/subclass
- ☐ wrapper class/primitive type
- ☐ primitive type/wrapper class
- ☐ subclass/superclass

Q5 Vocabulary/Theory

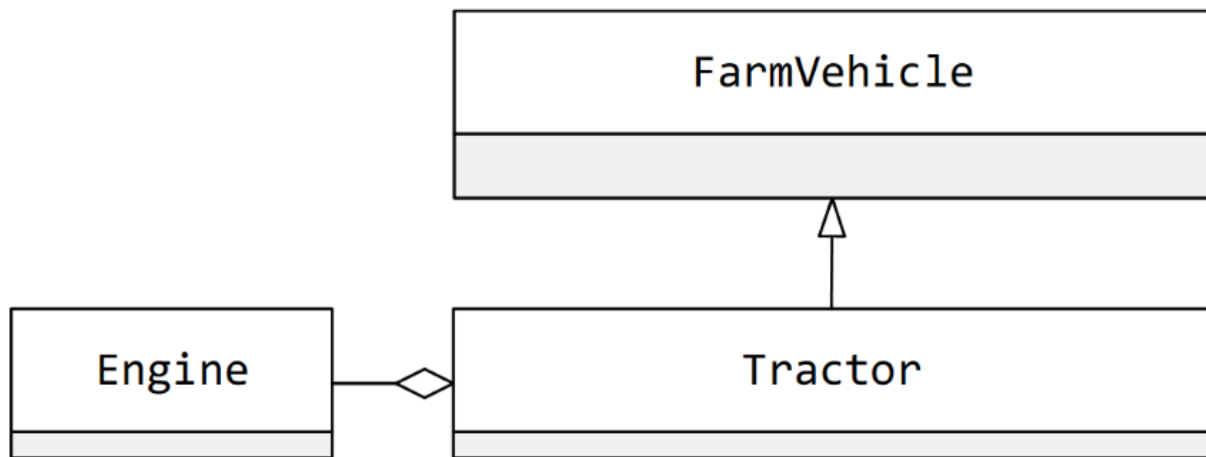
3 Points

What does the `super` keyword provide access to when used in a Java class?

- ☐ subclass members
- ☐ package members
- ☒ superclass members
- ☐ current class members

Q6 Reading UML/Java Code

11 Points



Use this simplified UML diagram to answer the following questions.

Q6.1 Reading UML/Java Code

3 Points

Provide the Java class header for the `FarmVehicle` class.

Expand the code to show the relationship to `Tractor`, if necessary.

Include the opening curly brace for the class and provide appropriate visibility for all items.

```
public class FarmVehicle {
```

Q6.2 Reading UML/Java Code

3 Points

Provide the Java class header for the `Engine` class.

Expand the code to show the relationship to `Tractor`, if necessary.

Include the opening curly brace for the class and provide appropriate visibility for all items.

```
public class Engine {
```

Q6.3 Reading UML/Java Code

5 Points

Provide the Java class header for the `Tractor` class.

Expand the code to show the relationship to `FarmVehicle` and `Engine`, if necessary.

Include the opening curly brace for the class and provide appropriate visibility for all items.

```
public class Tractor extends FarmVehicle {
```

Q7 Vocabulary/Theory

3 Points

`protected` visibility on a data member allows access to _____. Choose all correct answers.

☐ no one

☒ package

☐ subclass

☐ superclass

Q8 Vocabulary/Theory

3 Points

Where is the **default** implementation of `toString()` provided? Choose the most correct answer.

☐ By the developer

☒ Provided by the `Object` class

☐ Always defined by the parent class

☐ Auto-implemented by the JVM

Q9 Vocabulary/Theory

3 Points

Constructor chaining is ____ .

- ☐ having multiple constructors in a class to allow users to customize their instances
- ☒ when constructors call other constructors using the `this` or `super` keywords
- ☐ an automatic process used in inheritance for classes to obtain their superclass's properties
- ☐ using the constructors in a class to set specific object member values

Q10 Vocabulary/Theory

3 Points

In object-oriented programming, polymorphism _____. Choose the answer best suited for object-oriented programming.

- ☐ is the ability for a class to be referred to using the data types of any subclasses or related classes
- ☐ allows a class to be referred to using any data type at any time
- ☒ is the ability for a class to be referred to using the data types of its superclass type
- ☐ means "many forms" or "many shapes"

Q11 Vocabulary/Theory

3 Points

Text file I/O in Java requires the use of the ____ class for input, and the ____ class for output.

- ☒ `Scanner` / `PrintWriter`
- ☐ `InputFile` / `OutputFile`
- ☐ `System.in` / `System.out`
- ☐ `PrintWriter` / `Scanner`

Q12 Vocabulary/Theory

3 Points

What is the purpose of the `toString()` method? Choose the most correct answer.

- ☐ to provide some information to the developer
- ☒ to describe all data members and their values
- ☐ to provide a textual representation of the object
- ☐ to convert numeric data members into String values

Q13 Reading UML/Java Code

3 Points

`Motorcycle` is a subclass of another class named `TwoWheeledVehicle`. All `Motorcycle`s have an `Engine`, and `Engine`s can be purchased on their own.

Which of the following simplified UML diagrams correctly indicate the relationships between `TwoWheeledVehicle`, `Motorcycle`, and `Engine`?

- ☐

```
classDiagram
    class TwoWheeledVehicle
    class Motorcycle
    class Engine
    TwoWheeledVehicle <|-- Motorcycle
    Motorcycle *-- Engine
```

UML diagram showing `TwoWheeledVehicle` as a superclass of `Motorcycle` (indicated by a hollow triangle arrow pointing from `Motorcycle` to `TwoWheeledVehicle`). `Motorcycle` has a composition relationship with `Engine` (indicated by a solid line with a filled diamond at the `Motorcycle` end).
- ☒

```
classDiagram
    class TwoWheeledVehicle
    class Motorcycle
    class Engine
    TwoWheeledVehicle <|-- Motorcycle
    Motorcycle o-- Engine
```

UML diagram showing `TwoWheeledVehicle` as a superclass of `Motorcycle` (indicated by a hollow triangle arrow pointing from `Motorcycle` to `TwoWheeledVehicle`). `Motorcycle` has an aggregation relationship with `Engine` (indicated by a solid line with an open diamond at the `Motorcycle` end).
- ☐

```
classDiagram
    class TwoWheeledVehicle
    class Motorcycle
    class Engine
    TwoWheeledVehicle <|-- Motorcycle
    Motorcycle *-- Engine
```

UML diagram showing `TwoWheeledVehicle` as a superclass of `Motorcycle` (indicated by a hollow triangle arrow pointing from `Motorcycle` to `TwoWheeledVehicle`). `Motorcycle` has a composition relationship with `Engine` (indicated by a solid line with a filled diamond at the `Motorcycle` end).
- ☐

```
classDiagram
    class TwoWheeledVehicle
    class Motorcycle
    class Engine
    TwoWheeledVehicle <|-- Motorcycle
    Motorcycle *-- Engine
```

UML diagram showing `TwoWheeledVehicle` as a superclass of `Motorcycle` (indicated by a hollow triangle arrow pointing from `Motorcycle` to `TwoWheeledVehicle`). `Motorcycle` has a composition relationship with `Engine` (indicated by a solid line with a filled diamond at the `Motorcycle` end).

Q14 Vocabulary/Theory

3 Points

In object-oriented programming, dynamic binding ____ .

- ☒ allows the JVM to match method implementations to actual type at compile time
- ☐ allows the JVM to match method implementations to declared type at runtime
- ☐ allows the JVM to match method implementations to declared type at compile time
- ☐ allows the JVM to match method implementations to actual type at runtime

Q15 Vocabulary/Theory

3 Points

Inheritance models the "____" relationship between classes.

- ☐ has a
- ☐ is kind of
- ☐ is not like a
- ☒ is a

Q16 Vocabulary/Theory

3 Points

What is the main purpose of using exceptions?

- ☐ To allow text file input and output
- ☐ To prevent users from entering bad input into a running program
- ☒ To allow a program to continue to execute and/or recover from unexpected errors
- ☐ To allow a program to respond to user input in multiple ways

Q17 Vocabulary/Theory

3 Points

In Java, exception handling makes use of a ____/____ block to detect and handle
Exception S.

- ☒ try / catch
- ☐ catch / finally
- ☐ try / finally
- ☐ catch / exception

Q18 Vocabulary/Theory

3 Points

How is an ArrayList different from a "regular" array?

- ☐ Requires the user to specify a concrete type
- ☐ Allows access to convenience methods not accessible with regular arrays
- ☐ Can contain both reference and primitive types
- ☒ Dynamically sized to allow for easy insertion and removal

Q19 Vocabulary/Theory

3 Points

When working with text files, it is recommended to use try-with-resources. What **major** functionality does this mechanism provide?

- ☐ Allows exceptions in input to be caught and handled
- ☐ Allows programmers to create resources for use in input/output
- ☐ Allows programmers to read and write text files
- ☒ Allows for created resources to be auto-closed once the block is done executing